
FAULT TOLERANCE

A characteristic feature of distributed systems that distinguishes them from single-machine systems is the notion of partial failure: part of the system is failing while the remaining part continues to operate, and seemingly correctly. An important goal in distributed-systems design is to construct the system in such a way that it can automatically recover from partial failures without seriously affecting the overall performance. In particular, whenever a failure occurs, the system should continue to operate in an acceptable way while repairs are being made. In other words, a distributed system is expected to be fault tolerant.

In this chapter, we take a closer look at techniques to achieve fault tolerance. After providing some general background, we will first look at process resilience through process groups. In this case, multiple identical processes cooperate providing the appearance of a single logical process to ensure that one or more of them can fail without a client noticing. A specifically difficult point in process groups is reaching consensus among the group members on which a client-requested operation is to perform. By now, Paxos is a commonly adopted, yet relatively intricate algorithm, which we explain by building it from the ground up. Likewise, we carefully examine the cases in which consensus can be reached, and under which circumstances.

Achieving fault tolerance and reliable communication are strongly related. Next to reliable client-server communication we pay attention to reliable group communication and notably atomic multicasting. In the latter case, a message is delivered to all nonfaulty processes in a group, or to none at all. Having atomic multicasting makes development of fault-tolerant solutions much easier.

Atomicity is a property that is important in many applications. In this chapter, we pay attention to what are known as distributed commit protocols by which a group of processes are conducted to either jointly commit their local work, or collectively abort and return to a previous system state.

Finally, we will examine how to recover from a failure. In particular, we consider when and how the state of a distributed system should be saved to allow recovery to that state later on.

8.1 Introduction to fault tolerance

Fault tolerance has been subject to much research in computer science. In this section, we start with presenting the basic concepts related to processing failures, followed by a discussion of failure models. The key technique for handling failures is redundancy, which is also discussed. For more general information on fault tolerance in distributed systems, see, for example Jalote [1994]; Shooman [2002] or Koren and Krishna [2007].

Basic concepts

To understand the role of fault tolerance in distributed systems we first need to take a closer look at what it actually means for a distributed system to tolerate faults. Being fault tolerant is strongly related to what are called **dependable systems**. Dependability is a term that covers a number of useful requirements for distributed systems including the following [Kopetz and Verissimo, 1993]:

- Availability
- Reliability
- Safety
- Maintainability

Availability is defined as the property that a system is ready to be used immediately. In general, it refers to the probability that the system is operating correctly at any given moment and is available to perform its functions on behalf of its users. In other words, a highly available system is one that will most likely be working at a given instant in time.

Reliability refers to the property that a system can run continuously without failure. In contrast to availability, reliability is defined in terms of a time interval instead of an instant in time. A highly reliable system is one that will most likely continue to work without interruption during a relatively long period of time. This is a subtle but important difference when compared to availability. If a system goes down on average for one, seemingly random millisecond every hour, it has an availability of more than 99.9999 percent, but is still unreliable. Similarly, a system that never crashes but is shut down for two specific weeks every August has high reliability but only 96 percent availability. The two are not the same.

Safety refers to the situation that when a system temporarily fails to operate correctly, no catastrophic event happens. For example, many process-control systems, such as those used for controlling nuclear power plants or

sending people into space, are required to provide a high degree of safety. If such control systems temporarily fail for only a very brief moment, the effects could be disastrous. Many examples from the past (and probably many more yet to come) show how hard it is to build safe systems.

Finally, **maintainability** refers to how easily a failed system can be repaired. A highly maintainable system may also show a high degree of availability, especially if failures can be detected and repaired automatically. However, as we shall see later in this chapter, automatically recovering from failures is easier said than done.

Note 8.1 (More information: Traditional metrics)

We can be a bit more precise when it comes to describing availability and reliability. Formally, the availability $A(t)$ of a component in the time interval $[0, t]$ is defined as the average fraction of time that the component has been functioning correctly during that interval. The **long-term availability** A of a component is defined as $A(\infty)$.

Likewise, the reliability $R(t)$ of a component in the time interval $[0, t]$ is formally defined as the conditional probability that it has been functioning correctly during that interval given that it was functioning correctly at time $T = 0$. Following Pradhan [1996], to establish $R(t)$ we consider a system of N identical components. Let $N_0(t)$ denote the number of correctly operating components at time t and $N_1(t)$ the number of failed components. Then, clearly,

$$R(t) = \frac{N_0(t)}{N} = 1 - \frac{N_1(t)}{N} = \frac{N_0(t)}{N_0(t) + N_1(t)}$$

The rate at which components are failing can be expressed as the derivative $dN_1(t)/dt$. Dividing this by the number of correctly operating components at time t gives us the **failure rate function** $z(t)$:

$$z(t) = \frac{1}{N_0(t)} \frac{dN_1(t)}{dt}$$

From

$$\frac{dR(t)}{dt} = -\frac{1}{N} \frac{dN_1(t)}{dt}$$

it follows that

$$z(t) = \frac{1}{N_0(t)} \frac{dN_1(t)}{dt} = -\frac{N}{N_0(t)} \frac{dR(t)}{dt} = -\frac{1}{R(t)} \frac{dR(t)}{dt}$$

If we make the simplifying assumption that a component does not age (and thus essentially has no wear-out phase), its failure rate will be constant, i.e., $z(t) = z$, implying that

$$\frac{dR(t)}{dt} = -zR(t)$$

Because $R(0) = 1$, we obtain

$$R(t) = e^{-zt}$$

In other words, if we ignore aging of a component, we see that a constant failure rate leads to a reliability following an exponential distribution, having the form shown in Figure 8.1.

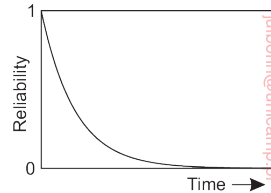


Figure 8.1: The reliability of a component having a constant failure rate.

Traditionally, fault-tolerance has been related to the following three metrics:

- **Mean Time To Failure (MTTF):** The average time until a component fails.
- **Mean Time To Repair (MTTR):** The average time needed to repair a component.
- **Mean Time Between Failures (MTBF):** Simply $MTTF + MTTR$.

Note that

$$A = \frac{MTTF}{MTBF} = \frac{MTTF}{MTTF + MTTR}$$

Also, these metrics make sense only if we have an accurate notion of what a failure actually is. As we will encounter later, identifying the occurrence of a failure may actually not be so obvious.

Often, dependable systems are also required to provide a high degree of security, especially when it comes to issues such as integrity. We will discuss security in the next chapter.

A system is said to **fail** when it cannot meet its promises. In particular, if a distributed system is designed to provide its users with a number of services, the system has failed when one or more of those services cannot be (completely) provided. An **error** is a part of a system's state that may lead to a failure. For example, when transmitting packets across a network, it is to be expected that some packets have been damaged when they arrive at the receiver. Damaged in this context means that the receiver may incorrectly sense a bit value (e.g., reading a 1 instead of a 0), or may even be unable to detect that something has arrived.

The cause of an error is called a **fault**. Clearly, finding out what caused an error is important. For example, a wrong or bad transmission medium may easily cause packets to be damaged. In this case, it is relatively easy to remove the fault. However, transmission errors may also be caused by bad weather conditions such as in wireless networks. Changing the weather to reduce or prevent errors is a bit trickier.

As another example, a crashed program is clearly a failure, which may have happened because the program entered a branch of code containing a programming bug (i.e., a programming error). The cause of that bug is

typically a programmer. In other words, the programmer is the cause of the error (programming bug), in turn leading to a failure (a crashed program).

Building dependable systems closely relates to controlling faults. As explained by Avizienis et al. [2004], a distinction can be made between preventing, tolerating, removing, and forecasting faults. For our purposes, the most important issue is **fault tolerance**, meaning that a system can provide its services even in the presence of faults. For example, by applying error-correcting codes for transmitting packets, it is possible to tolerate, to a certain extent, relatively poor transmission lines and reducing the probability that an error (a damaged packet) may lead to a failure.

Faults are generally classified as transient, intermittent, or permanent. **Transient faults** occur once and then disappear. If the operation is repeated, the fault goes away. A bird flying through the beam of a microwave transmitter may cause lost bits on some network (not to mention a roasted bird). If the transmission times out and is retried, it will probably work the second time.

An **intermittent fault** occurs, then vanishes of its own accord, then reappears, and so on. A loose contact on a connector will often cause an intermittent fault. Intermittent faults cause a great deal of aggravation because they are difficult to diagnose. Typically, when the fault doctor shows up, the system works fine.

A **permanent fault** is one that continues to exist until the faulty component is replaced. Burnt-out chips, software bugs, and disk-head crashes are examples of permanent faults.

Failure models

A system that fails is not adequately providing the services it was designed for. If we consider a distributed system as a collection of servers that communicate with one another and with their clients, not adequately providing services means that servers, communication channels, or possibly both, are not doing what they are supposed to do. However, a malfunctioning server itself may not always be the fault we are looking for. If such a server depends on other servers to adequately provide its services, the cause of an error may need to be searched for somewhere else.

Such dependency relations appear in abundance in distributed systems. A failing disk may make life difficult for a file server that is designed to provide a highly available file system. If such a file server is part of a distributed database, the proper working of the entire database may be at stake, as only part of its data may be accessible.

To get a better grasp on how serious a failure actually is, several classification schemes have been developed. One such scheme is shown in Figure 8.2, and is based on schemes described by Cristian [1991] and Hadzilacos and Toueg [1993].

Type of failure	Description of server's behavior
Crash failure	Halts, but is working correctly until it halts
Omission failure <i>Receive omission</i> <i>Send omission</i>	Fails to respond to incoming requests Fails to receive incoming messages Fails to send messages
Timing failure	Response lies outside a specified time interval
Response failure <i>Value failure</i> <i>State-transition failure</i>	Response is incorrect The value of the response is wrong Deviates from the correct flow of control
Arbitrary failure	May produce arbitrary responses at arbitrary times

Figure 8.2: Different types of failures.

A **crash failure** occurs when a server prematurely halts, but was working correctly until it stopped. An important aspect of crash failures is that once the server has halted, nothing is heard from it anymore. A typical example of a crash failure is an operating system that comes to a grinding halt, and for which there is only one solution: reboot it. Many personal computer systems suffer from crash failures so often that people have come to expect them to be normal. Consequently, moving the reset button from the back of a cabinet to the front was done for good reason. Perhaps one day it can be moved to the back again, or even removed altogether.

An **omission failure** occurs when a server fails to respond to a request. Several things might go wrong. In the case of a **receive-omission failure**, possibly the server never got the request in the first place. Note that it may well be the case that the connection between a client and a server has been correctly established, but that there was no thread listening to incoming requests. Also, a receive-omission failure will generally not affect the current state of the server, as the server is unaware of any message sent to it.

Likewise, a **send-omission failure** happens when the server has done its work, but somehow fails in sending a response. Such a failure may happen, for example, when a send buffer overflows while the server was not prepared for such a situation. Note that, in contrast to a receive-omission failure, the server may now be in a state reflecting that it has just completed a service for the client. As a consequence, if the sending of its response fails, the server has to be prepared for the client to reissue its previous request.

Other types of omission failures not related to communication may be caused by software errors such as infinite loops or improper memory management by which the server is said to “hang.”

Another class of failures is related to timing. **Timing failures** occur when the response lies outside a specified real-time interval. For example, in the case of streaming video's, providing data too soon may easily cause trouble for a recipient if there is not enough buffer space to hold all the incoming

data. More common, however, is that a server responds too late, in which case a *performance* failure is said to occur.

A serious type of failure is a **response failure**, by which the server's response is simply incorrect. Two kinds of response failures may happen. In the case of a value failure, a server simply provides the wrong reply to a request. For example, a search engine that systematically returns Web pages not related to any of the search terms used, has failed.

The other type of response failure is known as a **state-transition failure**. This kind of failure happens when the server reacts unexpectedly to an incoming request. For example, if a server receives a message it cannot recognize, a state-transition failure happens if no measures have been taken to handle such messages. In particular, a faulty server may incorrectly take default actions it should never have initiated.

The most serious are **arbitrary failures**, also known as **Byzantine failures**. In effect, when arbitrary failures occur, clients should be prepared for the worst. In particular, it may happen that a server is producing output it should never have produced, but which cannot be detected as being incorrect. Byzantine failures were first analyzed by Pease et al. [1980] and Lamport et al. [1982]. We return to such failures below.

Note 8.2 (More information: Omission and commission failures)

It has become somewhat of a habit to associate the occurrence of Byzantine failures with maliciously operating processes. The term "Byzantine" refers to the Byzantine Empire, a time (330–1453) and place (the Balkans and modern Turkey) in which endless conspiracies, intrigue, and untruthfulness were alleged to be common in ruling circles.

However, it may not be possible to detect whether an act was actually benign or malicious. Is a networked computer running a poorly engineered operating system that adversely affects the performance of other computers acting maliciously? In this sense, it is better to make the following distinction, which effectively excludes judgement:

- An **omission failure** occurs when a component fails to take an action that it should have taken.
- A **commission failure** occurs when a component takes an action that it should not have taken.

This difference, introduced by Mohan et al. [1983], also illustrates that a separation between dependability and security may at times be pretty difficult to make.

Many of the aforementioned cases deal with the situation that a process *P* no longer perceives any actions from another process *Q*. However, can *P* conclude that *Q* has indeed come to a halt? To answer this question, we need to make a distinction between two types of distributed systems:

- In an **asynchronous system**, no assumptions about process execution speeds or message delivery times are made. The consequence is that when process P no longer perceives any actions from Q, it cannot conclude that Q crashed. Instead, it may just be slow or its messages may have been lost.
- In a **synchronous system**, process execution speeds and message-delivery times are bounded. This also means that when Q shows no more activity when it is expected to do so, process P can rightfully conclude that Q has crashed.

Unfortunately, pure synchronous systems exist only in theory. On the other hand, simply stating that every distributed system is asynchronous also does not do just to what we see in practice and we would be overly pessimistic in designing distributed systems under the assumption that they are necessarily asynchronous. Instead, it is more realistic to assume that a distributed system is **partially synchronous**: most of the time it behaves as a synchronous system, yet there is no bound on the time that it behaves in an asynchronous fashion. In other words, asynchronous behavior is an exception, meaning that we can normally use timeouts to conclude that a process has indeed crashed, but that occasionally such a conclusion is false. In practice, this means that we will have to design fault-tolerant solutions that can withstand incorrectly detecting that a process halted.

In this context, halting failures can be classified as follows, from the least to the most severe (see also Cachin et al. [2011]). We let process P attempt to detect that process Q has failed.

- **Fail-stop failures** refer to crash failures that can be reliably detected. This may occur when assuming nonfaulty communication links and when the failure-detecting process P can place a worst-case delay on responses from Q.
- **Fail-noisy failures** are like fail-stop failures, except that P will only *eventually* come to the correct conclusion that Q has crashed. This means that there may be some a priori unknown time in which P's detections of the behavior of Q are unreliable.
- When dealing with **fail-silent failures**, we assume that communication links are nonfaulty, but that process P cannot distinguish crash failures from omission failures.
- **Fail-safe failures** cover the case of dealing with arbitrary failures by process Q, yet these failures are benign: they cannot do any harm.
- Finally, when dealing with **fail-arbitrary failures**, Q may fail in any possible way; failures may be unobservable in addition to being harmful to the otherwise correct behavior of other processes.

Clearly, having to deal with fail-arbitrary failures is the worst that can happen. As we shall discuss shortly, we can design distributed systems in such a way that they can even tolerate these types of failures.

Failure masking by redundancy

If a system is to be fault tolerant, the best it can do is to try to hide the occurrence of failures from other processes. The key technique for masking faults is to use redundancy. Three kinds are possible: information redundancy, time redundancy, and physical redundancy (see also Johnson [1995]). With **information redundancy**, extra bits are added to allow recovery from garbled bits. For example, a Hamming code can be added to transmitted data to recover from noise on the transmission line.

With **time redundancy**, an action is performed, and then, if need be, it is performed again. Transactions use this approach. If a transaction aborts, it can be redone with no harm. Another well-known example is retransmitting a request to a server when lacking an expected response. Time redundancy is especially helpful when the faults are transient or intermittent.

With **physical redundancy**, extra equipment or processes are added to make it possible for the system as a whole to tolerate the loss or malfunctioning of some components. Physical redundancy can thus be done either in hardware or in software. For example, extra processes can be added to the system so that if a small number of them crash, the system can still function correctly. In other words, by replicating processes, a high degree of fault tolerance may be achieved. We return to this type of software redundancy later in this chapter.

Note 8.3 (More information: Triple modular redundancy)

It is illustrative to see how redundancy has been applied in the design of electronic devices. Consider, for example, the circuit of Figure 8.3(a). Here signals pass through devices A, B, and C, in sequence. If one of them is faulty, the final result will probably be incorrect.

In Figure 8.3(b), each device is replicated three times. Following each stage in the circuit is a triplicated voter. Each voter is a circuit that has three inputs and one output. If two or three of the inputs are the same, the output is equal to that input. If all three inputs are different, the output is undefined. This kind of design is known as **triple modular redundancy (TMR)**.

Suppose that element A2 fails. Each of the voters, V1, V2, and V3 gets two good (identical) inputs and one rogue input, and each of them outputs the correct value to the second stage. In essence, the effect of A2 failing is completely masked, so that the inputs to B1, B2, and B3 are exactly the same as they would have been had no fault occurred.

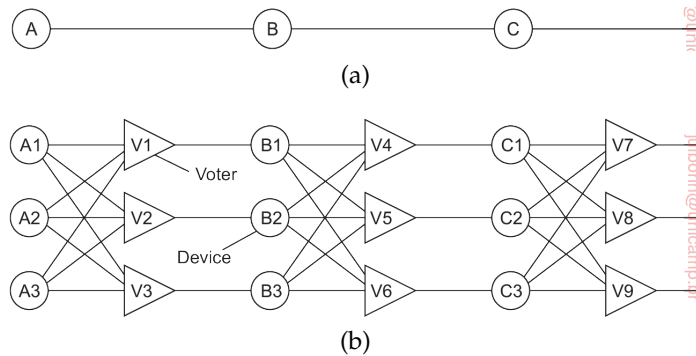


Figure 8.3: Triple modular redundancy.

Now consider what happens if B3 and C1 are also faulty, in addition to A2. These effects are also masked, so the three final outputs are still correct.

At first it may not be obvious why three voters are needed at each stage. After all, one voter could also detect and pass through the majority view. However, a voter is also a component and can also be faulty. Suppose, for example, that voter V1 malfunctions. The input to B1 will then be wrong, but as long as everything else works, B2 and B3 will produce the same output and V4, V5, and V6 will all produce the correct result into stage three. A fault in V1 is effectively no different than a fault in B1. In both cases B1 produces incorrect output, but in both cases it is voted down later and the final result is still correct.

Although not all fault-tolerant distributed systems use TMR, the technique is very general, and should give a clear feeling for what a fault-tolerant system is, as opposed to a system whose individual components are highly reliable but whose organization cannot tolerate faults (i.e., operate correctly even in the presence of faulty components). Of course, TMR can be applied recursively, for example, to make a chip highly reliable by using TMR inside it, unknown to the designers who use the chip, possibly in their own circuit containing multiple copies of the chips along with voters.

8.2 Process resilience

Now that the basic issues of fault tolerance have been discussed, let us concentrate on how fault tolerance can actually be achieved in distributed systems. The first topic we discuss is protection against process failures, which is achieved by replicating processes into groups. In the following pages, we consider the general design issues of process groups and discuss what a fault-tolerant group actually is. Also, we look at how to reach consensus within a process group when one or more of its members cannot be trusted to give correct answers.

Resilience by process groups

The key approach to tolerating a faulty process is to organize several identical processes into a group. The key property that all groups have is that when a message is sent to the group itself, all members of the group receive it. In this way, if one process in a group fails, hopefully some other process can take over for it [Guerraoui and Schiper, 1997].

Process groups may be dynamic. New groups can be created and old groups can be destroyed. A process can join a group or leave one during system operation. A process can be a member of several groups at the same time. Consequently, mechanisms are needed for managing groups and group membership.

The purpose of introducing groups is to allow a process to deal with collections of other processes as a single abstraction. Thus a process P can send a message to a group $Q = \{Q_1, \dots, Q_N\}$ of servers without having to know who they are, how many there are, or where they are, which may change from one call to the next. To P , the group Q appears to be a single, logical process.

Group organization

An important distinction between different groups has to do with their internal structure. In some groups, all processes are equal. There is no distinctive leader and all decisions are made collectively. In other groups, some kind of hierarchy exists. For example, one process is the coordinator and all the others are workers. In this model, when a request for work is generated, either by an external client or by one of the workers, it is sent to the coordinator. The coordinator then decides which worker is best suited to carry it out, and forwards it there. More complex hierarchies are also possible, of course. These communication patterns are illustrated in Figure 8.4.

Each of these organizations has its own advantages and disadvantages. The flat group is symmetrical and has no single point of failure. If one of the processes crashes, the group simply becomes smaller, but can otherwise continue. A disadvantage is that decision making is more complicated. For example, to decide anything, a vote often has to be taken, incurring some delay and overhead.

The hierarchical group has the opposite properties. Loss of the coordinator brings the entire group to a grinding halt, but as long as it is running, it can make decisions without bothering everyone else. In practice, when the coordinator in a hierarchical group fails, its role will need to be taken over and one of the workers is elected as new coordinator. We discussed leader-election algorithms in Chapter 6.

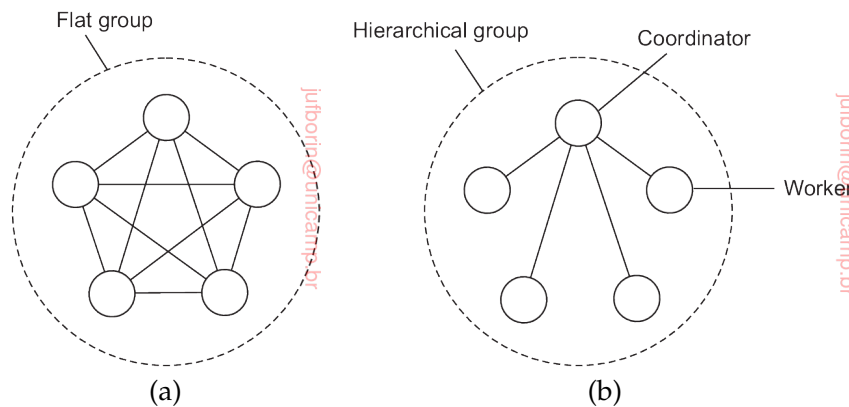


Figure 8.4: Communication in a (a) flat group and in a (b) hierarchical group.

Membership management

When group communication is present, some method is needed for creating and deleting groups, as well as for allowing processes to join and leave groups. One possible approach is to have a **group server** to which all these requests can be sent. The group server can then maintain a complete database of all the groups and their exact membership. This method is straightforward, efficient, and fairly easy to implement. Unfortunately, it shares a major disadvantage with all centralized techniques: a single point of failure. If the group server crashes, group management ceases to exist. Probably most or all groups will have to be reconstructed from scratch, possibly terminating whatever work was going on.

The opposite approach is to manage group membership in a distributed way. For example, if (reliable) multicasting is available, an outsider can send a message to all group members announcing its wish to join the group.

Ideally, to leave a group, a member just sends a goodbye message to everyone. In the context of fault tolerance, assuming fail-stop failure semantics is generally not appropriate. The trouble is, there is no polite announcement that a process crashes as there is when a process leaves voluntarily. The other members have to discover this experimentally by noticing that the crashed member no longer responds to anything. Once it is certain that the crashed member is really down (and not just slow), it can be removed from the group.

Another knotty issue is that leaving and joining have to be synchronous with data messages being sent. In other words, starting at the instant that a process has joined a group, it must receive all messages sent to that group. Similarly, as soon as a process has left a group, it must not receive any more messages from the group, and the other members must not receive any more messages from it. One way of making sure that a join or leave is integrated

into the message stream at the right place is to convert this operation into a sequence of messages sent to the whole group.

One final issue relating to group membership is what to do if so many processes go down that the group can no longer function at all. Some protocol is needed to rebuild the group. Invariably, some process will have to take the initiative to start the ball rolling, but what happens if two or three try at the same time? The protocol must be able to withstand this. Again, coordination through, for example, a leader-election algorithm may be needed.

Failure masking and replication

Process groups are part of the solution for building fault-tolerant systems. In particular, having a group of identical processes allows us to mask one or more faulty processes in that group. In other words, we can replicate processes and organize them into a group to replace a single (vulnerable) process with a (fault tolerant) group. As discussed in the previous chapter, there are two ways to approach such replication: by means of primary-based protocols, or through replicated-write protocols.

Primary-based replication in the case of fault tolerance generally appears in the form of a primary-backup protocol. In this case, a group of processes is organized in a hierarchical fashion in which a primary coordinates all write operations. In practice, the primary is fixed, although its role can be taken over by one of the backups, if need be. In effect, when the primary crashes, the backups execute some election algorithm to choose a new primary.

Replicated-write protocols are used in the form of active replication, as well as by means of quorum-based protocols. These solutions correspond to organizing a collection of identical processes into a flat group. The main advantage is that such groups have no single point of failure at the cost of distributed coordination.

An important issue with using process groups to tolerate faults is how much replication is needed. To simplify our discussion, let us consider only replicated-write systems. A system is said to be **k -fault tolerant** if it can survive faults in k components and still meet its specifications. If the components, say processes, fail silently, then having $k + 1$ of them is enough to provide k -fault tolerance. If k of them simply stop, then the answer from the other one can be used.

On the other hand, if processes exhibit arbitrary failures, continuing to run when faulty and sending out erroneous or random replies, a minimum of $2k + 1$ processes are needed to achieve k -fault tolerance. In the worst case, the k failing processes could accidentally (or even intentionally) generate the same reply. However, the remaining $k + 1$ will also produce the same answer, so the client or voter can just believe the majority.

Now suppose that in a k -fault tolerant group a single process fails. The group as a whole is still living up to its specifications, namely that it can

tolerate the failure of up to k of its members (of which one has just failed). But what happens if more than k members fail? In that case all bets are off and whatever the group does, its results, if any, cannot be trusted. Another way of looking at this is that the process group, in its appearance of mimicking the behavior of a single, robust process, has failed.

Consensus in faulty systems with crash failures

As mentioned, in terms of clients and servers, we have adopted a model in which a potentially very large collection of clients now send commands to a *group of processes* that jointly behave as a *single, highly robust process*. To make this work, we need to make an important assumption:

In a fault-tolerant process group, each nonfaulty process executes the same commands, in the same order, as every other nonfaulty process.

Formally, this means that the group members need to reach **consensus** on which command to execute. If failures cannot happen, reaching consensus is easy. For example, we can use Lamport's totally ordered multicasting as described in Section 6.2. Or, to keep it simple, using a centralized sequencer that hands out a sequence number to each command that needs to be executed will do the job as well. Unfortunately, life is not without failures, and reaching consensus among a group of processes under more realistic assumptions turns out to be tricky.

To illustrate the problem at hand, let us assume we have a group of processes $P = \{P_1, \dots, P_n\}$ operating under fail-stop failure semantics. In other words, we assume that crash failures can be reliably detected among the group members. Typically a client contacts a group member requesting it to execute a command. Every group member maintains a list of proposed commands: some which it received directly from clients; others which it received from its fellow group members. We can reach consensus using the following approach, adopted from Cachin et al. [2011], and referred to as **flooding consensus**.

Conceptually the algorithm operates in rounds. In each round, a process P_i sends its list of proposed commands it has seen so far to every other process in P . At the end of a round, each process merges all received proposed commands into a new list, from which it then will deterministically select the command to execute, if possible. It is important to realize that the selection algorithm is the same for all processes. In other words, if all process have exactly the same list, they will all select the same command to execute (and remove that command from their list).

It is not difficult to see that this approach works as long as processes do not fail. Problems start when a process P_i detects, during round r , that, say process P_k has crashed. To make this concrete, assume we have a process

group of four processes $\{P_1, \dots, P_4\}$ and that P_1 crashes during round r . Also, assume that P_2 receives the list of proposed commands from P_1 before it crashes, but that P_3 and P_4 do not (in other words, P_1 crashes before it got a chance to send its list to P_3 and P_4). This situation is sketched in Figure 8.5.

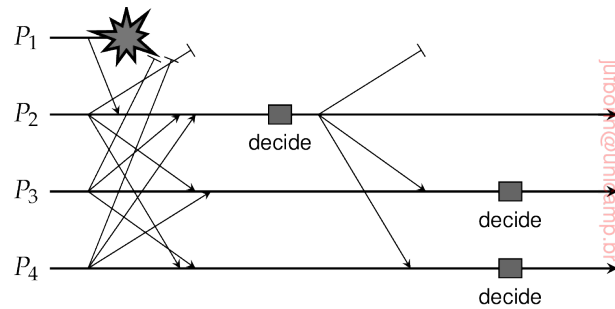


Figure 8.5: Reaching consensus through flooding in the presence of crash failures. Adopted from Cachin et al. [2011].

Assuming that all processes knew who was group member at the beginning of round r , P_2 is ready to make a decision on which command to execute when it receives the respective lists of the other members: it has all commands proposed so far. Not so for P_3 and P_4 . For example, P_3 may detect that P_1 crashed, but it does not know if either P_2 or P_4 had already received P_1 's list. From P_3 's perspective, if there is another process that did receive P_1 's proposed commands, that process may then make a different decision than itself. As a consequence, the best that P_3 can do is postpone its decision until the next round. The same holds for P_4 in this example. A process will decide to move to a next round when it has received a message from every nonfaulty process. This assumes that each process can reliably detect the crashing of another process, for otherwise it would not be able to decide who the nonfaulty processes are.

Because process P_2 received all commands, it can indeed make a decision and can subsequently broadcast that decision to the others. Then, during the next round $r + 1$, processes P_3 and P_4 will also be able to make a decision: they will decide to execute the same command selected by P_2 .

To understand why this algorithm is correct, it is important to realize that a process will move to a next round without having made a decision, only when it detects that another process has failed. In the end, this means that in the worst case at most one nonfaulty process remains, and this process can simply decide whatever proposed command to execute. Again, note that we are assuming reliable failure detection.

But then, what happens when the decision by process P_2 that it sent to P_3 was lost? In that case, P_3 can still not make a decision. Worse, we need to make sure that it makes the same decision as P_2 and P_4 . If P_2 did not

crash, we can assume that a retransmission of its decision will save the day. If P_2 did crash, this will be also detected by P_4 who will then subsequently rebroadcast its decision. In the meantime, P_3 has moved to a next round, and after receiving the decision by P_4 , will terminate its execution of the algorithm.

Example: Paxos

The flooding-based consensus algorithm is not very realistic if only for the fact that it relies on a fail-stop failure model. More realistic is to assume a fail-noisy failure model in which a process will *eventually* reliably detect that another process has crashed. In the following, we describe a simplified version of a widely adopted consensus algorithm, known as **Paxos**. It was originally published in 1989 as a technical report by Leslie Lamport, but it took about a decade before someone decided that it may not be such a bad idea to disseminate it through a regular scientific channel [Lamport, 1998]. The original publication is not easy to understand, exemplified by other publications that aim at explaining it [Lampson, 1996; Prisco et al., 1997; Lamport, 2001; van Renesse and Altinboken, 2015].

Essential Paxos

The assumptions under which Paxos operates are rather weak:

- The distributed system is partially synchronous (in fact, it may even be asynchronous).
- Communication between processes may be unreliable, meaning that messages may be lost, duplicated, or reordered.
- Messages that are corrupted can be detected as such (and thus subsequently ignored).
- All operations are deterministic: once an execution is started, it is known exactly what it will do.
- Processes may exhibit crash failures, but not arbitrary failures, nor do processes collude.

By-and-large, these are realistic assumptions for many practical distributed systems.

We roughly follow the explanation given by Lamport [2001] and Kirsch and Amir [2008]. The algorithm operates as a network of *logical* processes, of which there are different types. First, there are **clients** that request a specific operation to be executed. At the server side, each client is represented by a single **proposer**, which attempts to have a client's request accepted. Normally, a single proposer has been designated as being the **leader**, and drives the protocol toward reaching consensus.

What we need to establish is that a proposed operation is accepted by an **acceptor**. If a majority of acceptors accepts the same proposal, the proposal is said to be *chosen*. However, what is chosen still needs to be *learned*. To this end, we will have a number of **learner** processes, each of which will execute a chosen proposal once it has been informed by a majority of acceptors.

It is important to note that a single proposer, acceptor, and learner form a single *physical* process, running on a single machine that the client communicates with, as shown in Figure 8.6. We thus assume that if, for example, a proposer crashes, then the physical process that it is part of will have crashed. By replicating this server we aim at obtaining fault tolerance in the presence of crash failures.

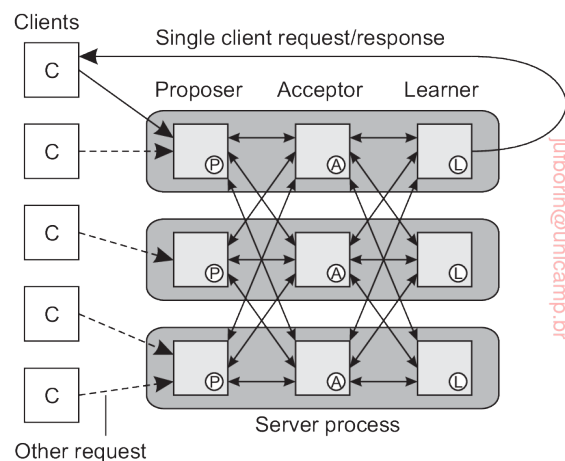


Figure 8.6: The organization of Paxos into different logical processes.

The basic model is that the leading proposer receives requests from clients, one at a time. A nonleading proposer forwards any client request to the leader. The leading proposer sends its proposal to all acceptors, telling each to accept the requested operation. Each acceptor will subsequently broadcast a learn message. If a learner receives the same learn message from a majority of acceptors, it knows that consensus has been reached on which operation to execute, and will execute it.

There are at least two specific issues that need further attention. First, not only do the servers need to reach consensus on which operation to execute, we also need to make sure that each of them actually executes it. In other words, how do we know for sure that a majority of the nonfaulty servers will carry out the operation? There is essentially only one way out: have learn messages be retransmitted. However, to make this work, an acceptor will have to log its decisions (in turn requiring a mechanism for purging logs). Because we are assuming globally ordered proposal timestamps (as explained shortly), missing messages can be easily detected, and also accepted operations will

always be executed in the same order by all learners.

As a general rule, the server hosting the leading proposer will also inform the client when its requested operation has been executed. If another process had taken over the lead, then it will also handle the response to the client.

This brings us to the second important issue: a failing leader. Life would be easy if the failure of a leader would be reliably detected, after which a new leader would be elected, and later, the recovering leader would instantly notice that the world around it had changed. Unfortunately, life is not so easy. Paxos has been designed to tolerate proposers who still believe they are in the lead. The effect is that proposals may be sent out concurrently by different proposers (each believing to be the leader). We therefore need to make sure that these proposals can be distinguished from one another in order to ensure that the acceptors handle only the proposals from the current leader.

Note that relying on a leading proposer implies that any practical implementation of Paxos will need to be accompanied by a leader-election algorithm. In principle, that algorithm can operate independently from Paxos, but is normally part of it.

To distinguish concurrent proposals from different proposers, each proposal p has a uniquely associated (logical) timestamp $ts(p)$. How uniqueness is achieved is left to an implementation, but we will describe some of the details shortly. Let $oper(p)$ denote the operation associated with proposal p . The trick is to allow multiple proposals to be accepted, but that each of these accepted proposals has the same associated operation. This can be achieved by guaranteeing that if a proposal p is chosen, then any proposal with a higher timestamp will also have the same associated operation. In other words, we require that

$$p \text{ is chosen} \Rightarrow \text{for all } p' \text{ with } ts(p') > ts(p) : oper(p') = oper(p)$$

Of course, for p to be chosen, it needs to be accepted. That means that we can guarantee our requirement when guaranteeing that if p is chosen, then any higher-timestamped proposal accepted by any acceptor, has the same associated operation as p . However, this is not sufficient, for suppose that at a certain moment a proposer simply sends a new proposal p' , with the highest timestamp so far, to an acceptor A that had not received any proposal before. Note that this may indeed happen according to our assumptions concerning message loss and multiple proposers each believing to be in the lead. In absence of any other proposals, A will simply accept p' . To prevent this situation from happening, we thus need to guarantee that

If proposal p is chosen, then any higher-timestamped proposal issued by a proposer, has the same associated operation as p .

When explaining the Paxos algorithm below, we will indeed see that a proposer may need to adopt an operation coming from acceptors in favor of its

own. This will happen after a leading proposer had failed, but its proposed operation had already made it to a majority of the acceptors.

The processes collectively formally ensure *safety*, in the sense that only proposed operations will be learned, and that at most one operation will be learned at a time. In general, a **safety property** asserts that nothing bad will happen. Furthermore, Paxos ensures *conditional liveness* in the sense that if enough processes remain up-and-running, then a proposed operation will eventually be learned (and thus executed). **Liveness**, which tells us that eventually something good will happen, is not guaranteed in Paxos, unless some adaptations are made. We return to liveness in Note 8.4.

There are now two phases, each in turn consisting of two subphases. During the first phase, the leading proposer interacts with acceptors to get a requested operation accepted for execution. The first phase is needed to rule out any trouble caused by different proposers each believing they are the leader. The best that can happen is that an individual acceptor promises to consider the proposer's operation and ignore other requests. The worst is that the proposer was too late and that it will be asked to adopt some other proposer's request instead. Apparently, a leadership change had taken place and there may be former requests that need to be handled first.

In the second phase, the acceptors will have informed proposers about the promises they have made. The leading proposer essentially takes up a slightly different role by promoting a single operation to the one to be executed, and subsequently telling the acceptors.

Phase 1a (prepare): The goal of this phase is that a proposer P who believes it is leader and is proposing operation o , tries to get its proposal timestamp **anchored**, in the sense that any lower timestamp failed, or that o had also been previously proposed (i.e., with some lower proposal timestamp). To this end, P broadcasts its proposal to the acceptors. For the operation o , the proposer selects a proposal number m higher than any of its previously selected numbers. This leads to a **proposal timestamp** $t = (m, i)$ where i is the (numerical) process identifier of P . Note that

$$(m, i) < (n, j) \Leftrightarrow (m < n) \text{ or } (m = n \text{ and } i < j)$$

This timestamp for a proposal p is an implementation of the previously mentioned timestamp $ts(p)$. Proposer P sends $\text{PREPARE}(t)$ to all acceptors (but note that messages may be lost). In doing so, it is (1) asking the acceptors to promise not to accept any proposals with a lower proposal timestamp, and (2) to inform it about an accepted proposal, if any, with the highest timestamp less than t . If such a proposal exists, the proposer will adopt it.

Phase 1b (promise): An acceptor A may receive multiple proposals. Assume it receives $\text{PREPARE}(t)$ from P . There are three cases to consider:

- (1) t is the highest proposal timestamp received from any proposer so far. In that case, A will return `PROMISE( $t$ )` to P stating that A will ignore any future proposals with a lower timestamp.
- (2) If t is the highest timestamp so far, but another proposal (t', o') had already been accepted, A also returns (t', o') to P. This will allow P to decide on the final operation that needs to be accepted.
- (3) In all other cases, do nothing: there is apparently another proposal with a higher timestamp that is being processed.

Once the first phase has been completed, the leading proposer P knows what the acceptors have promised. Essentially, the leading proposer knows that all acceptors have agreed on the same operation. This will put P into a position to tell the acceptors that they can go ahead. This is needed, because although the leading proposer knows on which operation consensus has been reached, this consensus is not known to the others. Again, we assume that P received a response from a majority of acceptors (whose respective responses may be different).

Phase 2a (accept): There are two cases to consider:

- (1) If P does not receive any accepted operation from any of the acceptors, it will forward its own proposal for acceptance by sending `ACCEPT( $t, o$ )` to all acceptors.
- (2) Otherwise, it was informed about another operation o' , which it will adopt and forward for acceptance. It does so by sending `ACCEPT( $t, o'$ )`, where t is P's proposal timestamp and o' is the operation with proposal timestamp highest among all accepted operations that were returned by the acceptors in Phase 1b.

Phase 2b (learn): Finally, if an acceptor receives `ACCEPT( $t, o'$ )`, but did not previously send a promise with a higher proposal timestamp, it will accept operation o' , and tell all learners to execute o' by sending `LEARN( $o'$ )`. At that point, the acceptor can forget about o' . A learner L receiving `LEARN( $o'$ )` from a majority of acceptors, will execute the operation o' . We now also know that a majority of learners share the same idea on which operation to execute.

It is important to realize that this description of Paxos indeed captures only its essence: using a leading proposer to drive the acceptors toward the execution of the same operation. When it comes to practical implementations, much more needs to be done (and more than we are willing to describe here). An excellent description of what it means to realize Paxos has been written by Kirsch and Amir [2008]. Another write-up on its practical implications can be found in [Chandra et al., 2007].

Understanding Paxos

To properly understand Paxos, but also many other consensus algorithms, it is useful to see how its design could have evolved. We say “could have,” as the evolution of the algorithm has never been documented. The following description is largely based on work described by Meling and Jehl [2013]¹. As our starting point, we consider a server that we wish to make more robust. By now, we know that this can be achieved through replication and making sure that all commands submitted by clients are executed by all servers in the same order. The simplest situation is to add one server, thus creating a group of two processes, say S_1 and S_2 . Also, to make sure that all commands are executed in the same order, we appoint one process to be a sequencer, which increments and associates a unique timestamp with every submitted command. Servers are required to execute commands according to their timestamp. In Paxos, such a server is referred to as the **leader**. We can also consider it to be a **primary server**, with the other acting as a **backup server**.

We assume that a client broadcasts its requested command to all servers. If a server notices it is missing a command, it can rely on the other server to forward it when necessary. We will not describe how this happens, but silently assume that all commands are stored at the servers and that we merely need to make sure that the servers agree on which command to execute next. As a consequence, all remaining communication between servers consists of control messages. To make this point clear, consider the situation sketched in Figure 8.7. (In what follows, we use subscripts to designate processes, and superscripts to designate operations and states.)

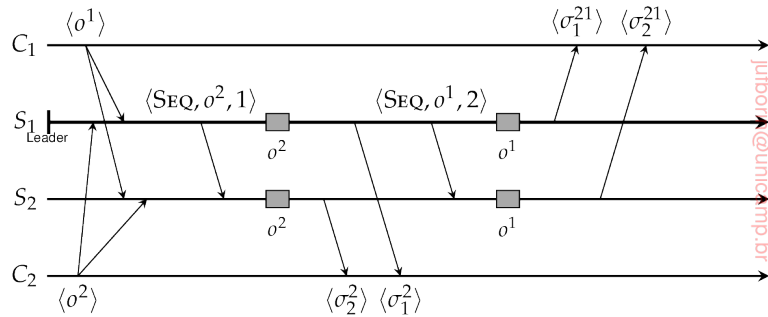


Figure 8.7: Two clients communicating with a 2-server process group.

In this example, server S_1 is the leader and as such will hand out timestamps to submitted requests. Client C_1 has submitted command o^1 while C_2 submitted o^2 . S_1 instructs S_2 to execute operation o^2 with timestamp 1, and later operation o^1 with timestamp 2. After processing a command, a server will return the result to the associated client. We designate this using the

¹Special credits go to Hein Meling for helping us better understand what Paxos is all about.

notation $\langle \sigma_i^j \rangle$, where i is the index of the reporting server, and j the state it was in, expressed as the sequence of operations it has carried out. In our example, client C_1 will thus see the result $\langle \sigma_k^{21} \rangle$, meaning that each server has executed o^1 after executing o^2 .

In Paxos, when a leader associates a timestamp with an operation, it does so by sending an **accept message** to the other server(s). As we assume that messages may be lost, a server accepting an operation o does so by telling the leader it has **learned** the operation by returning a **LEARN(o)** message. When the leader does not notice that operation o has been learned, it simply retransmits an **ACCEPT(o, t)** message, with t being the original timestamp. Note that in our description we are skipping the phase of coming to agreement on the operation to be carried out: we assume the leader has decided and now needs to reach consensus on executing that operation.

Compensating for a lost message is relatively easy, but what happens when also a server crashes? Let us first assume that a crash can be reliably detected. Consider the situation sketched in Figure 8.8(a). The issue, of course, is that server S_2 will never have learned (about) operation o^1 . This situation can be prevented by demanding that a server may execute an operation only if it knows that the other server has learned the operation as well, as illustrated in Figure 8.8(b).

It is not difficult to see that with a larger process group, we can get into the same situation as in Figure 8.8(a). Simply consider a group of three servers $\{S_1, S_2, S_3\}$ with S_1 being the leader. If its **ACCEPT(o^1, t)** message to S_3 is lost, yet it knows that S_2 has learned o^1 , then it should still not execute o^1 until it has also received a **LEARN(o^1)** message from S_3 . This situation is shown in Figure 8.9.

Now imagine what would happen if **LEARN(o^1)** returned by S_2 would not make it to S_1 . Of course, S_1 would not execute o^1 , but otherwise we would still be in trouble: S_2 will execute o^1 while S_3 would take over leadership and execute o^2 without ever knowing that o^1 had already been processed. In other words, also S_2 must wait with the execution of o^1 until it knows that S_3 has learned that operation. This brings us to the following:

*In Paxos, a server S cannot execute an operation o until it has received a **LEARN(o)** from all other nonfaulty servers.*

Up to this point we have assumed that a process can reliably detect that another process has crashed. In practice, this is not the case. As we will discuss more extensively shortly, a standard approach toward failure detection is to set a timeout on expected messages. For example, each server is required to send a message declaring it is still alive, and at the same time the other servers set timeouts on the expected receipt of such messages. If a timeout expires, the sender is suspected to have failed. In a partially synchronous or fully asynchronous system, there is essentially no other solution. However,

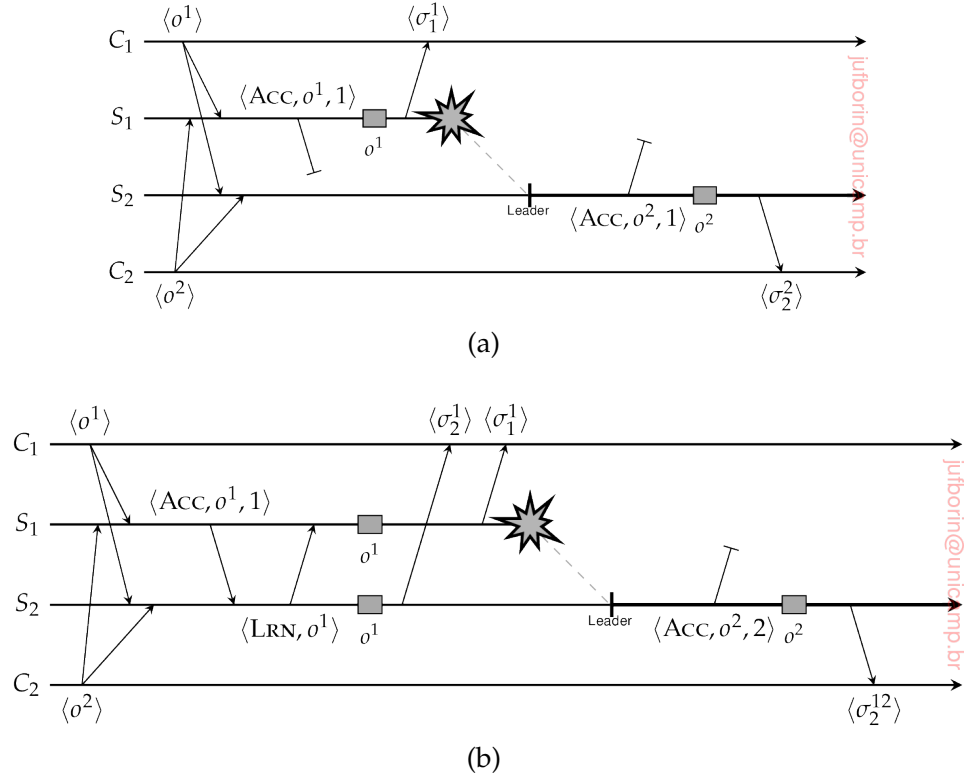


Figure 8.8: (a) What may happen when the leader crashes in combination with a lost accept, and (b) the solution, namely demanding that the other server has learned the operation as well before executing it.

the consequence is that a failure may be falsely detected, as the delivery of such “I’m alive” messages may have simply been delayed or lost.

Let us assume that Paxos has realized a failure detection mechanism, but that the two servers falsely conclude that the other has failed, as shown in Figure 8.10. The problem is clear: each may now independently decide to execute their operation of choice, leading to divergent behavior. It is at this point that we need to introduce an extra server, and demand that a server can execute an operation only if it is certain that a majority will execute that operation. Note, that in the three-server case, execution of operation o by server S can take place as soon as S has received at least one (other) $\text{LEARN}(o)$ message. Together with the sender of that message, S will form a majority.

We have now come to a point where it should be clear that Paxos requires at least three replicated servers $\{S_1, S_2, S_3\}$ in order to operate correctly. Let us concentrate on the situation that one of these servers crashes. We make the following assumptions.

that execution of an operation can take place only if a server knows that a majority will execute that operation.

Suppose now that S_1 crashes after the execution of operation o^1 . The worst that can happen in this case is that S_3 is completely ignorant of the situation until the new leader, S_2 tells it to accept operation o^2 . Note that this is announced through an $\text{ACCEPT}(o^2, 2)$ message such that the timestamp $t = 2$ will alert S_3 that it missed a previous accept message. S_3 will tell so to S_2 , who can then retransmit $\text{ACCEPT}(o^1, 1)$, allowing S_3 to catch up.

Likewise, if S_2 missed $\text{ACCEPT}(o^1, 1)$, but did detect that S_1 crashed, it will eventually either send $\text{ACCEPT}(o^1, 1)$ or $\text{ACCEPT}(o^2, 1)$ (i.e., in both cases using timestamp $t = 1$, which was previously used by S_1). Again, S_3 has enough information to get S_2 on the right track again. If S_2 had sent $\text{ACCEPT}(o^1, 1)$, S_3 can simply tell S_2 that it already learned o^1 . In the other case, when S_2 sends $\text{ACCEPT}(o^2, 1)$, S_3 will inform S_2 that it apparently missed operation o^1 . We conclude that when S_1 crashes after executing an operation, Paxos behaves correctly.

So what can happen if S_1 crashes immediately after having sent $\text{ACCEPT}(o^1, 1)$ to the other two servers? Suppose again that S_3 is completely ignorant of the situation because messages are lost, until S_2 has taken over leadership and announces that o^2 should be accepted. Like before, S_3 can tell S_2 that it (i.e., S_3) missed operation o^1 , so that S_2 can help S_3 to catch up. If S_2 misses messages, but does detect that S_1 crashed, then as soon as it takes over leadership and proposes an operation, it will be using a stale timestamp. This will trigger S_3 to inform S_2 that it missed operation o^1 , which saves the day. Again, Paxos is seen to behave correctly.

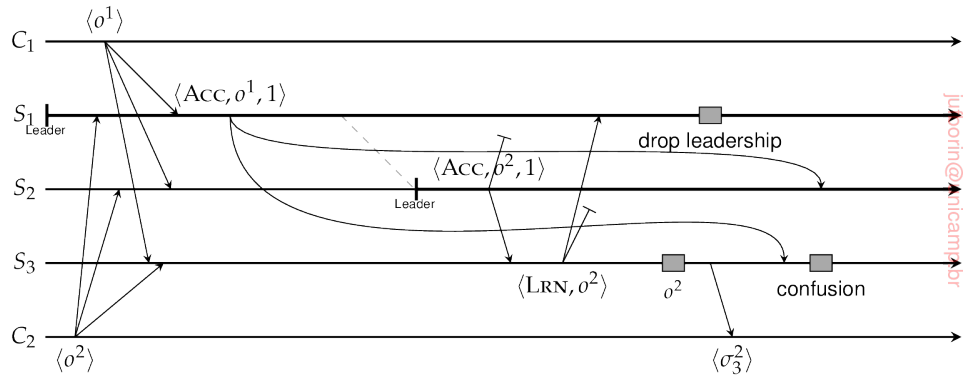


Figure 8.11: Why incorporating the ID of the current leader is needed.

Problems may arise with false detections of crashes. Consider the situation sketched in Figure 8.11. We see that the accept messages from S_1 are considerably delayed and that S_2 falsely detects S_1 having crashed. S_2 takes over leadership and sends $\text{ACCEPT}(o^2, 1)$, i.e., with a timestamp $t = 1$.

However, when finally $\text{ACCEPT}(o^1, 1)$ arrives, S_3 cannot do anything: this is not a message it is expecting. Fortunately, if it knows who the current leader is, in combination with a deterministic leader election, it could safely reject $\text{ACCEPT}(o^1, 1)$, knowing that by now S_2 has taken over. We conclude that the leader should include its ID in an accept message.

We have covered almost all cases and have thus far shown that Paxos behaves correctly. Unfortunately, although being correct, the algorithm can still come to a grinding halt. Consider the situation illustrated in Figure 8.12. What we are seeing here is that, because the learn messages returned by S_3 are lost, neither S_1 nor S_2 will ever be able to know what S_3 actually executed: did it learn (and execute) $\text{ACCEPT}(o^1, 1)$ before or after learning $\text{ACCEPT}(o^2, 1)$, or perhaps it learned neither operation? A solution to this problem is discussed in Note 8.4.

TBD: Correct behavior of S_3 : it should sent a message to S_2 that the timestamp is wrong. See mail Hermann 28-01-2020.

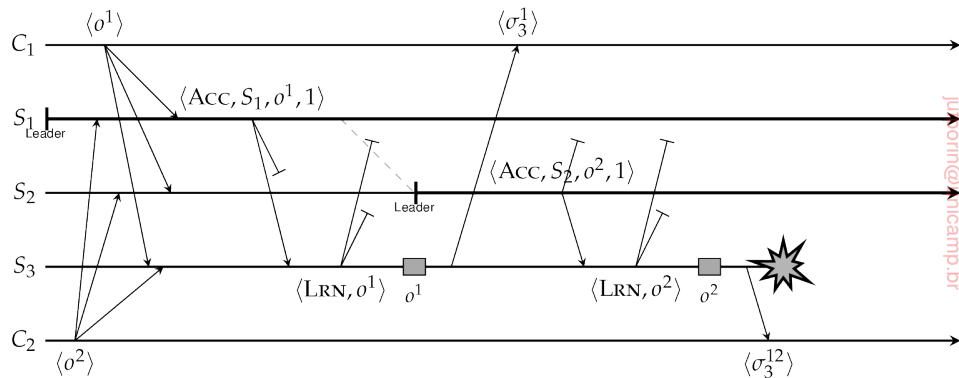


Figure 8.12: When Paxos can make no further progress.

Note 8.4 (Advanced: Making progress in Paxos)

Up to this point we have discussed the development of Paxos such that **safety** is ensured. Safety essentially means that nothing bad will happen, or, put differently, that the behavior of the algorithm is correct. To also ensure that eventually something good will happen, generally referred to as **liveness** of an algorithm, we need to do a bit more. In particular, we need to get out of the situation sketched in Figure 8.12.

The real problem with this situation is that the servers have no consensus on who the leader is. Once S_2 decides it should take over leadership, it needs to ensure that any outstanding operations initiated by S_1 have been properly dealt with. In other words, it needs to ensure that its own leadership is not hindered by operations that have not yet been completed by all nonfaulty processes. If leadership is taken over too quickly and a new operation is proposed, a previous